

JAVA DEVELOPMENT

ESTRUTURA DE REPETIÇÃO E STRINGS



LISTA DE FIGURAS

Figura 1 – Exercício da média da turma.....	10
Figura 2 – Execução do exercício da média da turma	11
Figura 3 – Ajuste do exercício da média da turma	12
Figura 4 – Execução do exercício com notas fora do intervalo	12
Figura 5 – Adicionando um breakpoint na linha 8 do programa intervalo.....	13
Figura 6 – Botão para executar o programa em modo debug	13
Figura 7 – Programa executando em modo debug	14
Figura 8 – Botões para executar as instruções em modo debug	14
Figura 9 – Aba do Console para ver e inserir as informações.....	15
Figura 10 – Execução do programa em modo debug e as variáveis com valores	15
Figura 11 – Botão Resume Program.....	16
Figura 12 – Solução possível para o exercício da média da turma	16
Figura 13 – Exemplo de sequência de escape.....	18
Figura 14 – Exemplo de concatenação de strings (1)	19
Figura 15 – Exemplo de concatenação de strings (2)	20
Figura 16 – Exemplo de concatenação de strings (3)	20
Figura 17 – Exemplo de comparação de strings (1)	21
Figura 18 – Ilustração do armazenamento das strings na memória.....	22
Figura 19 – Ilustração do armazenamento das strings no pool de strings	22
Figura 20 – Exemplo de comparação de strings (2)	23
Figura 21 – Exemplo de comparação de strings (3)	24
Figura 22 – Exemplo de comparação de strings (4)	24
Figura 23 – Exemplo de comparação de strings (5)	25
Figura 24 – Exemplo recuperando a quantidade de caractere de uma string	26
Figura 25 – Index dos caracteres de uma string	27
Figura 26 – Exemplo da utilização do método indexOf	27
Figura 27 – Exemplo de verificação de nome completo	28
Figura 28 – Exemplo do método indexOf utilizando uma palavra	28
Figura 29 – Exemplo do método lastIndexOf	29
Figura 30 – Exemplo do método subString	29
Figura 31 – Implementação do exemplo do método subString	30
Figura 32 – Método subString com um parâmetro	30
Figura 33 – Implementação do exemplo do método subString com um parâmetro ..	31
Figura 34 – Implementação do exemplo com o método subString e indexOf	32
Figura 35 – Implementação do exemplo com os métodos toUpperCase e toLowerCase	33
Figura 36 – Implementação do exemplo com o método replace	34
Figura 37 – Implementação do exercício proposto.....	35

LISTA DE TABELAS

Tabela 1 – Sequência de escape	18
--------------------------------------	----

EMEND

SUMÁRIO

ESTRUTURA DE REPETIÇÃO E STRINGS.....	5
1 COMENTÁRIOS.....	5
2 ESTRUTURAS DE REPETIÇÃO	5
2.1 For	6
2.2 While	7
2.3 Do-while	8
2.4 Strings	17
REFERÊNCIAS.....	36

ESTRUTURA DE REPETIÇÃO E STRINGS

1 COMENTÁRIOS

No Java, assim como em outras linguagens, existem os comentários, textos inseridos dentro do código-fonte que não alteram o funcionamento do programa. Servem para detalhar alguma implementação e ajudar os outros desenvolvedores a entenderem o código. Mas utilize-os com moderação, porque no começo do aprendizado é interessante comentar o código para facilitar o entendimento. Porém, no nível profissional, não é necessário comentar todo o código, pois um código bem escrito não necessita de comentários para dizer o que está fazendo.

Existem dois tipos de comentários no Java: o de linha e o de bloco. O comentário de linha, como o próprio nome já diz, comenta uma linha. Já o de bloco adiciona uma ou mais linhas de comentário.

Para comentar uma linha, utilizamos duas barras (//). Já o de bloco se inicia com uma barra e um asterisco (/*) e finaliza com asterisco e barra (*/).

Exemplos:

//Exemplo de comentário de linha

**/* Exemplo
de comentário
de bloco */**

2 ESTRUTURAS DE REPETIÇÃO

Imagine que você precise executar um mesmo código várias vezes. Como escrever os números de 1 a 100 no console ou somar as médias dos 30 alunos da turma para calcular a média de notas da turma. Em vez de escrever o comando de escrita de dados `System.out.println()` cem vezes ou escrever 30 vezes a soma das médias dos alunos no programa, podemos escrever algumas poucas linhas de código. Além disso, imagine se, em vez de cem, fosse preciso escrever mil vezes? Ou se a quantidade de alunos da turma se alterasse de 30 para 20 ou 50?

Com as estruturas de repetição ou loops, essas tarefas serão extremamente simples. Porque permitem que um bloco de código seja executado repetidamente, enquanto alguma condição seja atendida. O que facilita a implementação, mesmo que seja necessário alterar a quantidade de vezes que o bloco deve ser executado.

2.1 For

A estrutura de repetição **for** é controlada por uma variável numérica que será o contador, atualizada depois de cada iteração. Essa estrutura é usada quando sabemos a quantidade de vezes que desejamos executar.

Sintaxe básica:

```
for ( <inicialização> ; <condição lógica> ; <incremento ou decremento> ) {  
    //instruções  
}
```

A primeira parte da estrutura, **<inicialização>**, é utilizada para definir e inicializar a variável de controle. A segunda parte, **<condição lógica>**, determina se o laço de repetição deve ou não ser executado novamente. E a última parte, **<incremento ou decremento>**, deve atualizar a variável de controle, o que é realizado após cada iteração.

Exemplo:

```
for (int i = 0; i <= 100; i++){  
    System.out.println(i);  
}
```

Este exemplo imprime os valores de 0 a 100. Primeiro, foi definido dentro da estrutura do **for** a variável **i**, do tipo **int** e inicializada com 0. A condição lógica é **i <= 100**, ou seja, enquanto a variável **i** for menor ou igual a 100, o loop será executado. Por fim, a variável **i** é incrementada em 1 a cada iteração.

2.2 While

A estrutura de repetição **while** (enquanto) executa até que a condição seja falsa, ou seja, enquanto a condição for verdadeira.

Estrutura básica do comando:

```
while (<condição lógica>) {  
  
    //instruções  
  
}
```

O loop corre o risco de nunca executar caso a condição seja falsa logo de início. O exemplo abaixo também imprime os números de 0 a 100:

Exemplo:

```
int i = 0;  
  
while (i <= 100) {  
    System.out.println(i);  
    i++;  
}
```

No exemplo, declaramos uma variável **i** do tipo **int** e inicializamos com o valor 0. Depois, a estrutura de repetição valida se o valor de **i** é menor ou igual a 100. Como a condição é verdadeira, o **while** é executado, imprimindo o valor de **i** e incrementando-o. Na próxima validação, o **i** é igual a 1 e o resultado da condição também é verdadeira, e o processo se repete até que o **i** seja igual a 101, quando a condição do **while** se torna falsa e o **while** para de ser executado.

Como vimos, o **for** e o **while** podem resolver o mesmo problema. Porém, a estrutura de repetição **while** é indicada para quando não sabemos exatamente a quantidade de vezes que o loop será executado. Por exemplo, podemos criar um programa Java para ler os dados de vários produtos, mas não sabemos quantos produtos exatamente o usuário informará. Assim, podemos perguntar, no final do cadastro do produto, se o usuário deseja cadastrar mais um produto, criando uma condição lógica para o loop se repetir.

Exemplo:

```
Scanner leitor = new Scanner(System.in);

int op = 1;

while (op == 1) {

    //instruções para ler os dados de um produto

    System.out.println("Deseja cadastrar mais um produto? 1-Sim 0-
Não");

    op = leitor.nextInt();

}
```

Não sabemos a quantidade de produtos que o usuário cadastrará, mas não tem problema, já que o programa está preparado para repetir o código de cadastro enquanto o usuário desejar.

Perceba que o **while** valida a parte inicial do loop, antes da execução da primeira iteração. Existe uma outra estrutura de repetição que valida no final. Assim, é garantido que o bloco da estrutura de repetição seja executado pelo menos uma vez. Essa estrutura é chamada de **do-while**.

2.3 Do-while

Essa estrutura de repetição é muito parecida com o **while**. A diferença é o lugar da verificação do loop, que, neste caso, é no final. A estrutura primeiro executa o bloco de código para depois testar a condição de repetição.

Sintaxe básica:

```
do {

    //instruções

} while (<condição>);
```

Perceba que, diferente do **while**, o **do-while** executa pelo menos uma vez o seu bloco de código, pois a condição é testada somente no final.

Exemplo:


```
int i = 0;

do {

    System.out.println(i);

    i++;

} while ( i <= 100);
```

E da mesma forma que o **while**, o **do-while** também é usado quando não sabemos exatamente quantas repetições serão efetuadas. Abaixo, temos o mesmo resultado do exemplo da estrutura **while**.

Exemplo:

```
Scanner leitor = new Scanner(System.in);

int op;

do {

    //instruções para ler os dados de um produto

    System.out.println("Deseja cadastrar mais um produto? 1-Sim 0-
Não");

    op = leitor.nextInt();

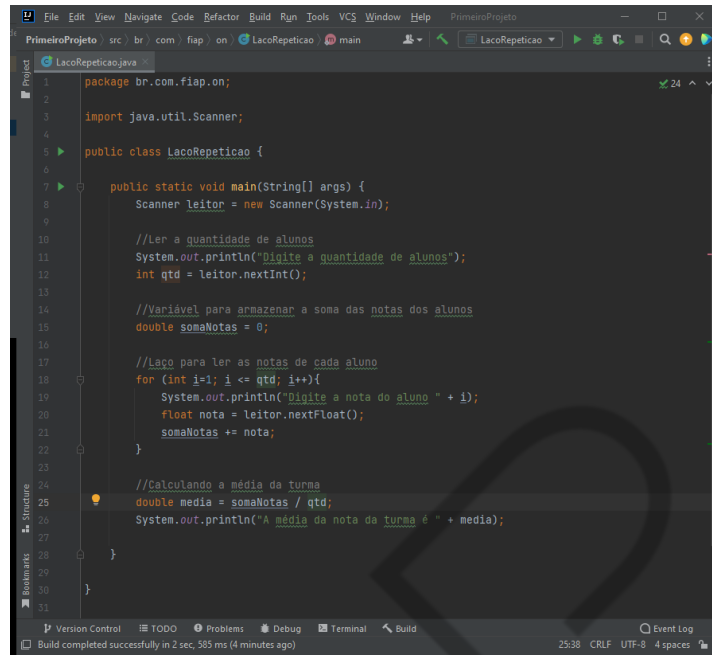
} while (op == 1);
```

Neste exemplo, foi declarada a variável **op** do tipo **int** e não foi atribuído nenhum valor a ela. Diferentemente do exemplo anterior, em que foi atribuído o valor 1. Isso é possível, pois, antes de testar o valor da variável, foi requisitado para o usuário inserir um número inteiro para depois validar o valor se é igual a 1.

A estrutura **do-while** se encaixa melhor neste exemplo, pois queremos sempre ler pelo menos um produto para depois perguntar se o usuário deseja cadastrar mais produtos.

Agora crie um programa Java para ler o número de alunos de uma turma. Depois, crie uma estrutura para ler as notas de cada aluno, de 0 a 10. Valide se a nota está dentro do intervalo. Por fim, exiba a média da turma.

Estrutura de repetição e Strings

The image shows a screenshot of an IDE (IntelliJ IDEA) with a Java file named 'LacoRepeticao.java'. The code is as follows:

```
1 package br.com.fiap.on;
2
3 import java.util.Scanner;
4
5 public class LacoRepeticao {
6
7     public static void main(String[] args) {
8         Scanner leitor = new Scanner(System.in);
9
10        //Ler a quantidade de alunos
11        System.out.println("Digite a quantidade de alunos");
12        int qtd = leitor.nextInt();
13
14        //Variável para armazenar a soma das notas dos alunos
15        double somaNotas = 0;
16
17        //Laço para ler as notas de cada aluno
18        for (int i=1; i <= qtd; i++){
19            System.out.println("Digite a nota do aluno " + i);
20            float nota = leitor.nextFloat();
21            somaNotas += nota;
22        }
23
24        //Calculando a média da turma
25        double media = somaNotas / qtd;
26        System.out.println("A média da nota da turma é " + media);
27    }
28 }
29
30
31 }
```

The IDE interface includes a menu bar at the top, a toolbar, and a sidebar on the left showing the project structure. The bottom status bar indicates the build was completed successfully.

Figura 1 – Exercício da média da turma
Fonte: Elaborado pelo autor (2022)

Análise o código apresentado na Figura “Exercício da média da turma”. Dentro do método **main**, na linha 8, criamos uma variável para armazenar o objeto capaz de ler o teclado do usuário. Depois, na linha 12, declaramos uma variável para armazenar a quantidade de alunos da turma e pedimos para o usuário inserir a informação.

Na linha 15, declaramos uma variável para armazenar a soma das notas de todos os alunos. Assim como sabemos a quantidade de alunos, é possível calcular a média da turma, dividindo a soma de todas as notas pela quantidade de alunos.

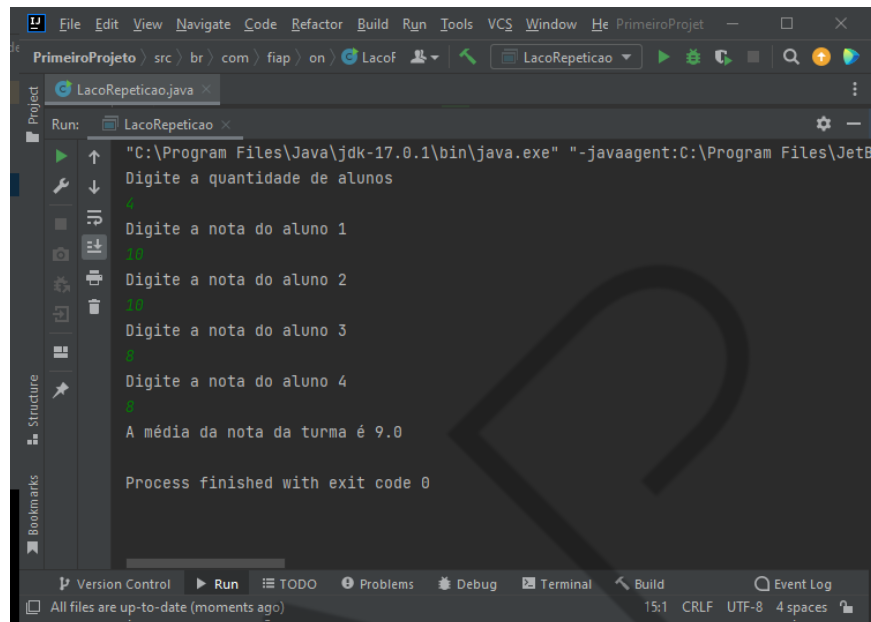
Agora, o laço de repetição! É possível resolver o problema com qualquer estrutura que foi vista (**while**, **do-while** ou **for**). Como sabemos a quantidade de vezes que queremos ler a nota do aluno, vamos optar pelo **for**. Observe que a variável de controle foi inicializada com 1 e não 0, pois vamos imprimir o valor de 1 a cada iteração, para dizer qual o número do aluno que estamos lendo a nota. E como foi inicializada com 1, a condição deve ser **i menor ou igual à quantidade de alunos**.

Na linha 19, exibimos uma informação para o usuário digitar a nota do aluno, junto do número do aluno. Depois, lemos a nota do aluno e, por fim, somamos à variável **somaNotas**.

Estrutura de repetição e Strings

Depois do laço de repetição, calculamos a média e exibimos o resultado.

Abaixo, apresentamos um resultado possível na execução do programa:



```
"C:\Program Files\Java\jdk-17.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetB
Digite a quantidade de alunos
4
Digite a nota do aluno 1
10
Digite a nota do aluno 2
10
Digite a nota do aluno 3
8
Digite a nota do aluno 4
8
A média da nota da turma é 9.0
Process finished with exit code 0
```

Figura 2 – Execução do exercício da média da turma
Fonte: Elaborado pelo autor (2022)

Você deve ter percebido que a implementação está incompleta. O usuário pode informar a nota fora do intervalo estipulado, ou seja, menor do que 0 ou maior do que 10. Vamos ajustar o programa para validar a nota antes de somá-la à variável de somatório. Caso esteja fora do intervalo, vamos pedir para inserir a nota novamente. Para isso, basta modificar o valor da variável de controle.

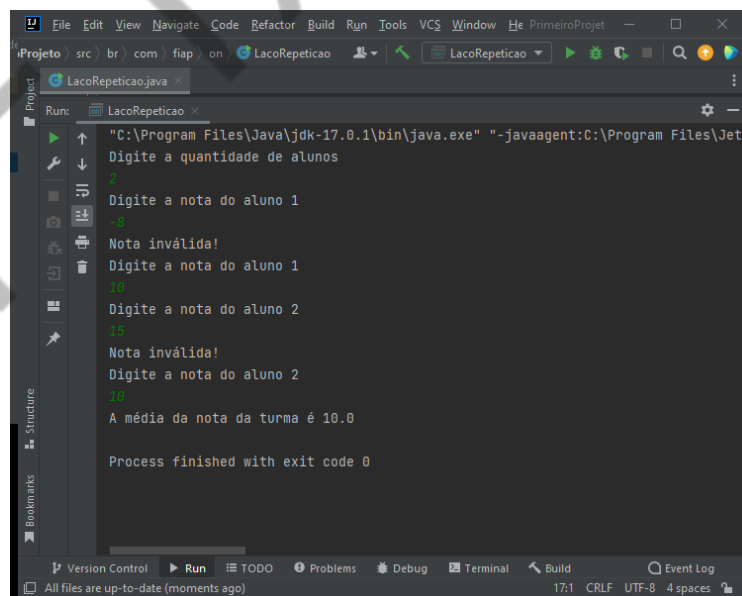
Estrutura de repetição e Strings



```
1 package br.com.fiap.on;
2
3 import java.util.Scanner;
4
5 public class LacoRepeticao {
6
7     public static void main(String[] args) {
8         Scanner leitor = new Scanner(System.in);
9
10        //Ler a quantidade de alunos
11        System.out.println("Digite a quantidade de alunos");
12        int qtd = leitor.nextInt();
13
14        //Variável para armazenar a soma das notas dos alunos
15        double somaNotas = 0;
16
17        //Laco para ler as notas de cada aluno
18        for (int i=1; i <= qtd; i++){
19            System.out.println("Digite a nota do aluno " + i);
20            float nota = leitor.nextFloat();
21            if (nota < 0 || nota > 10){
22                System.out.println("Nota inválida!");
23                i--;
24            } else {
25                somaNotas += nota;
26            }
27        }
28
29        //Calculando a média da turma
30        double media = somaNotas / qtd;
31        System.out.println("A média da nota da turma é " + media);
32    }
33}
34
35
36
```

Figura 3 – Ajuste do exercício da média da turma
Fonte: Elaborado pelo autor (2022)

Neste caso, caso a nota esteja fora do intervalo, o usuário é avisado e a variável de controle retorna ao valor anterior para ler novamente a nota do aluno.



```
Run: LacoRepeticao
"C:\Program Files\Java\jdk-17.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetB
Digite a quantidade de alunos
1
Digite a nota do aluno 1
15
Nota inválida!
Digite a nota do aluno 1
10
Digite a nota do aluno 2
15
Nota inválida!
Digite a nota do aluno 2
10
A média da nota da turma é 10.0
Process finished with exit code 0
```

Figura 4 – Execução do exercício com notas fora do intervalo
Fonte: Elaborado pelo autor (2022)

Estrutura de repetição e Strings

Para entender melhor a execução do código, podemos executar o programa em modo de debug, que executará o programa até parar em um “breakpoint”, ponto de parada determinado por nós. Para isso, basta dar um clique ao lado do número da linha no IntelliJ.

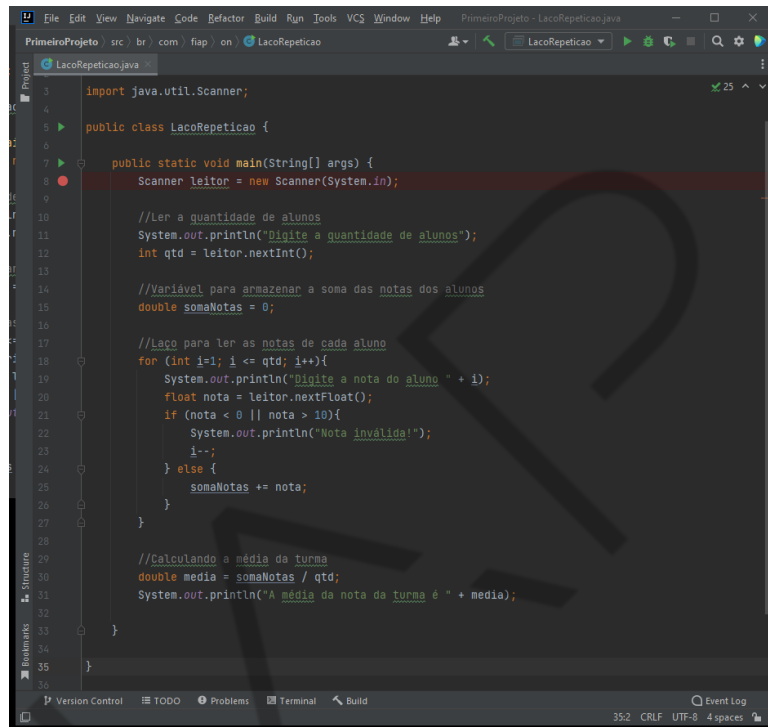


Figura 5 – Adicionando um breakpoint na linha 8 do programa intervalo
Fonte: Elaborado pelo autor (2022)

Você pode adicionar quantos breakpoints quiser no programa. Assim, quando executá-lo, o programa parará na linha marcada. Depois disso, você pode verificar o valor de todas as variáveis e executar as instruções linha a linha, facilitando o entendimento do programa. Esta é uma ferramenta muito importante para os desenvolvedores, pois permite encontrar problemas (bugs) mais facilmente.

Para executar o programa em modo debug, clique no botão do lado do “play”, que parece um inseto (bug), ou use o atalho SHIFT + F9.

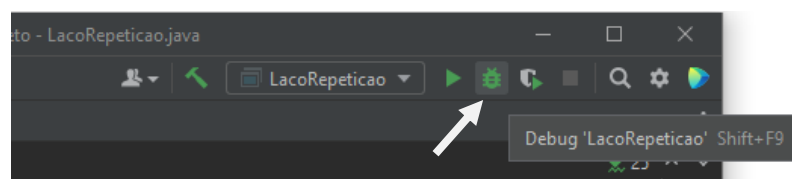


Figura 6 – Botão para executar o programa em modo debug
Fonte: Elaborado pelo autor (2022)

Estrutura de repetição e Strings

Quando executar em modo debug, o programa parará no primeiro breakpoint configurado no programa. Uma janela com as variáveis e seus valores será aberta, assim como os botões para prosseguir na execução do programa.

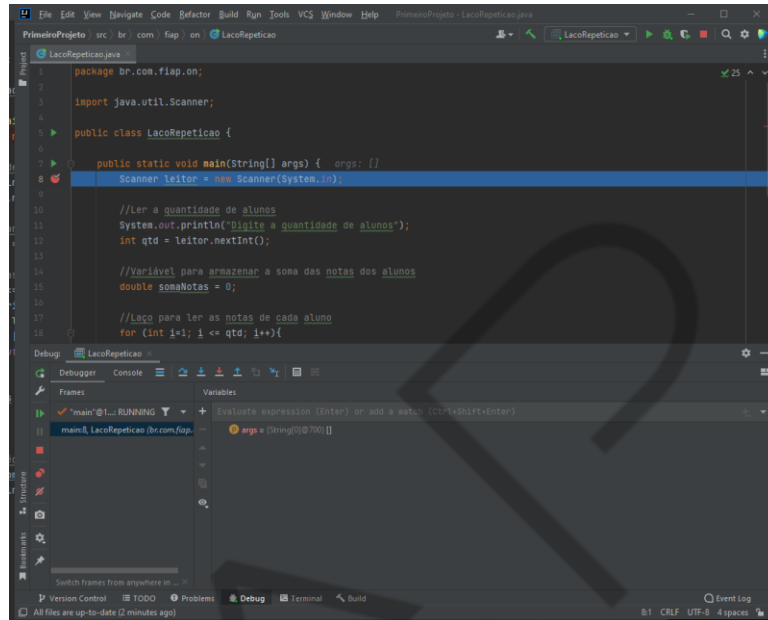


Figura 7 – Programa executando em modo debug
Fonte: Elaborado pelo autor (2022)

Você pode utilizar os botões ou os atalhos para continuar a execução do programa. O primeiro é o step over (F8), que executa a próxima linha; o segundo é o step into (F7), que também executa a próxima linha. A diferença entre eles é que, caso a instrução a ser executada seja uma chamada a um método, o step over não “entrará” dentro do método para executar linha a linha, já o step into executará o método chamado linha a linha. Mas não se preocupe, quando abordarmos os métodos voltaremos ao assunto. Então, por enquanto, vamos usar o step over (F8) para executar o programa. Faça o processo e veja o que está acontecendo a cada instrução executada.

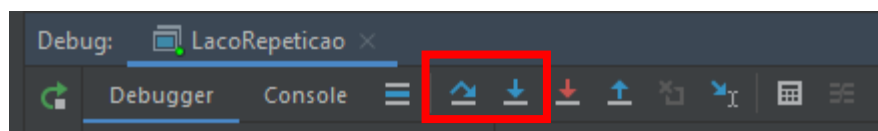


Figura 8 – Botões para executar as instruções em modo debug
Fonte: Elaborado pelo autor (2022)

Quando chegar à linha 12, onde o usuário deve inserir o número de alunos, clique na aba “Console” para inserir o valor.

Estrutura de repetição e Strings

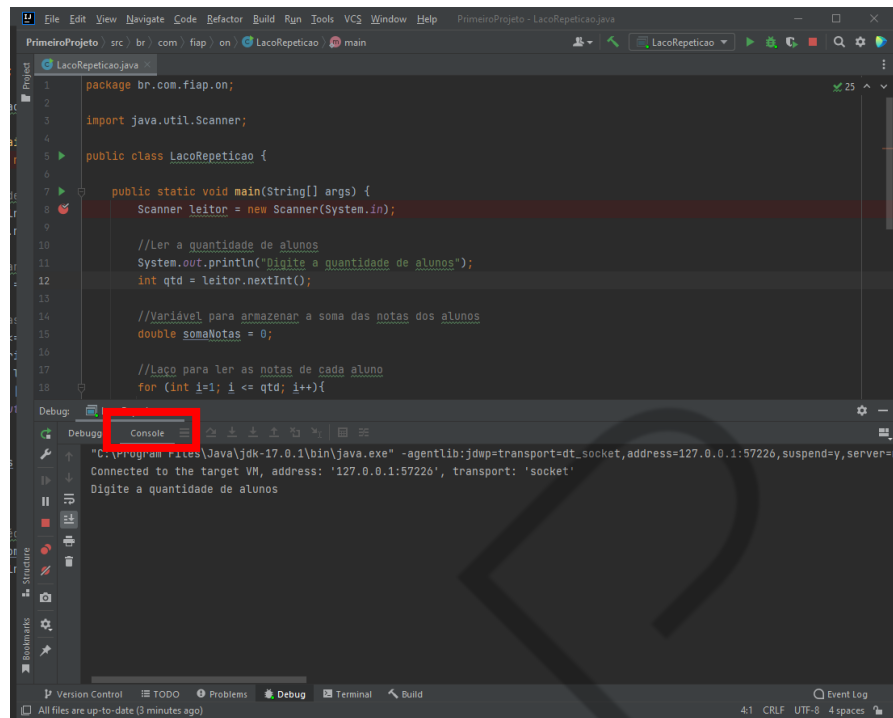


Figura 9 – Aba do Console para ver e inserir as informações
Fonte: Elaborado pelo autor (2022)

Veja que legal! Você pode compreender o que o programa está fazendo e ainda ver os valores das variáveis no momento da execução do programa.

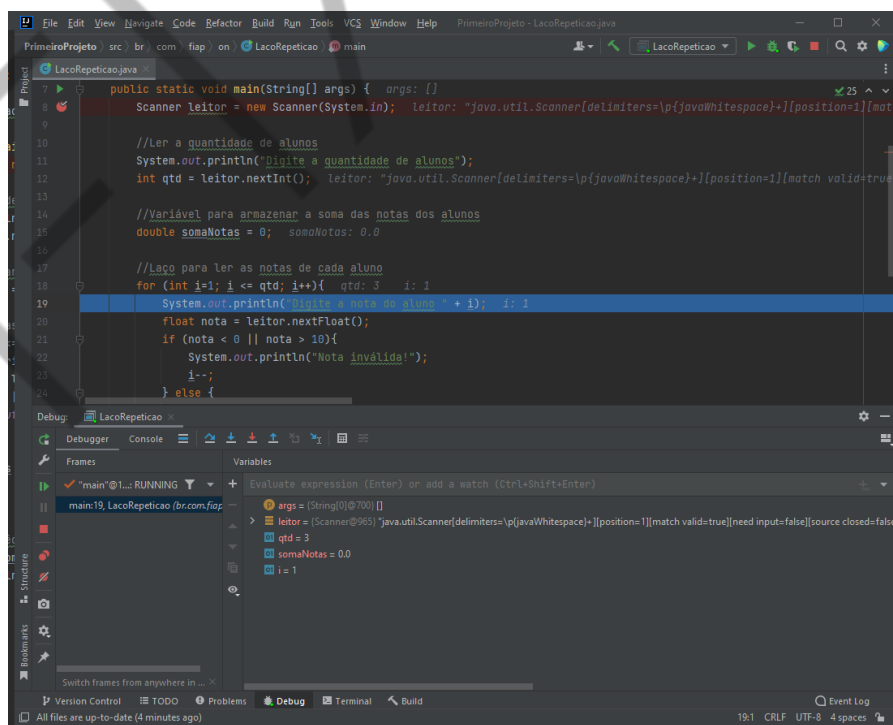


Figura 10 – Execução do programa em modo debug e as variáveis com valores
Fonte: Elaborado pelo autor (2022)

Estrutura de repetição e Strings

Aproveite e realize o debug dos outros programas desenvolvidos até aqui. Tenho certeza de que esta ferramenta será muito útil.

Caso queira executar o programa até o próximo breakpoint, ou até o final se não existir nenhum outro breakpoint configurado, utilize o atalho F9 ou o botão de Resume Program.

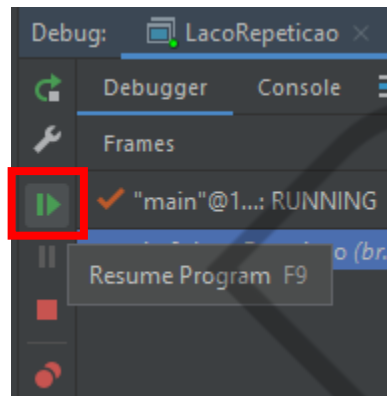


Figura 11 – Botão Resume Program
Fonte: Elaborado pelo autor (2022)

Agora vamos evoluir o programa das médias da turma. Não vamos pedir o número de alunos, iremos ler a nota do aluno e depois perguntar se o usuário deseja adicionar mais uma nota de um aluno. No final, diremos a média e a quantidade de alunos da turma. Bora lá!

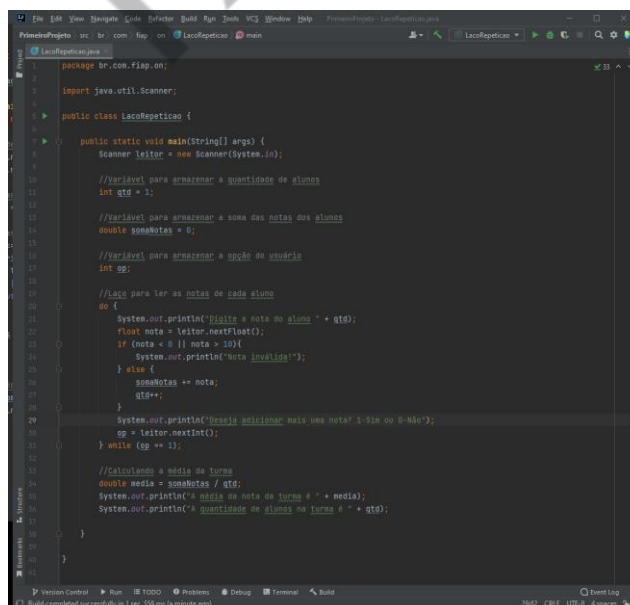


Figura 12 – Solução possível para o exercício da média da turma
Fonte: Elaborado pelo autor (2022)

Para resolver um problema, existem várias soluções. E isso acontece também com a lógica de programação. Neste exemplo, foi declarada uma variável chamada **qtd** para armazenar a quantidade de alunos. A estrutura escolhida foi a do-while, pois não conhecemos a quantidade de alunos no início e queremos ter pelo menos uma nota de um aluno. A variável **op** deve ser declarada fora do escopo do do-while, pois ela é utilizada fora dele, fora das chaves ({ }) da estrutura de repetição.

Nós só incrementamos a quantidade de alunos e somamos a nota à somatória de notas caso ela esteja dentro do intervalo estipulado. Depois, solicitamos ao usuário digitar 0 ou 1 para identificar se deseja ou não adicionar mais uma nota.

2.4 Strings

Para trabalhar com um conjunto de caracteres, temos as Strings no Java. Vimos o tipo primitivo char, que é capaz de armazenar um único caractere. Para armazenar textos e palavras, vamos usar uma classe chamada String, que faz parte da biblioteca-padrão da linguagem Java.

Os objetos strings no Java são imutáveis, isto é, seu conteúdo não pode ser alterado após a sua inicialização. Dessa forma, é impossível alterar o valor de uma string, porém é possível armazenar outra string no lugar da original.

Declarar uma variável do tipo String é igual à declaração de uma variável do tipo primitivo que vimos anteriormente. A diferença é que os tipos primitivos começam com a letra minúscula. Já a String, como é uma classe, deve começar com a letra maiúscula. Assim, declaramos o tipo do dado e o nome da variável, conforme o exemplo:

String nome;

Neste caso, o tipo de dado utilizado é “String” e o nome da variável é “nome”. Depois de declarar uma variável, podemos atribuir um valor. Outra diferença em relação ao tipo char é que ele define um único caractere com aspas simples. Enquanto as Strings, que são um conjunto de caracteres, devem ter os seus valores definidos com aspas duplas.

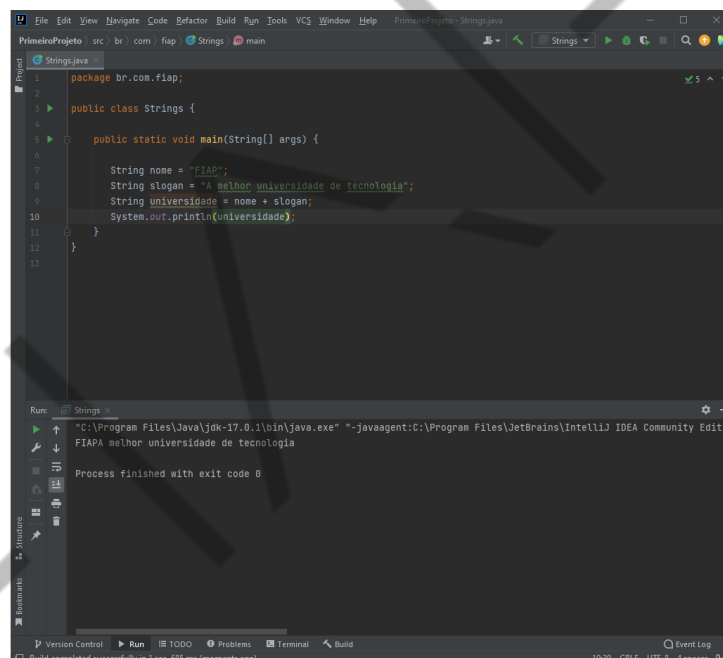
String nome = “Thiago”;

Estrutura de repetição e Strings

Assim, no exemplo, é possível ver a utilização da sequência “\t” antes da palavra FIAP para adicionar um espaço no começo da linha (tabulação). Depois, temos o “\n” para acrescentar uma quebra de linha e os “\” para adicionar aspas duplas antes e depois da frase “A melhor universidade de tecnologia”. É possível ver também o resultado da execução na Figura “Exemplo de sequência de escape”.

Assim como os números, podemos somar as strings. Porém, com as strings, a soma resulta na junção de uma ou mais strings, formando uma nova string. Essa operação também é conhecida como concatenação.

A forma mais simples de concatenar strings é por meio do operador de soma (+).



```
package br.com.fiap;

public class Strings {

    public static void main(String[] args) {

        String nome = "FIAP";
        String slogan = "A melhor universidade de tecnologia";
        String universidade = nome + slogan;
        System.out.println(universidade);
    }
}
```

Run: Strings

"C:\Program Files\Java\jdk-17.0.1\bin\java.exe" -javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Editio
FIAPA melhor universidade de tecnologia

Process finished with exit code 0

Figura 14 – Exemplo de concatenação de strings (1)

Fonte: Elaborado pelo autor (2022)

Primeiro, foram declaradas duas variáveis com as strings “FIAP” e “A melhor universidade de tecnologia”. Depois, declaramos uma terceira variável com o nome “universidade” e atribuímos os valores das duas outras variáveis concatenadas. O resultado impresso também pode ser visto na imagem: “FIAP A melhor universidade de tecnologia”.

As duas strings foram “somadas”, ou seja, foram ligadas. Porém, para ter um resultado melhor, podemos adicionar um espaço entre as strings:

Estrutura de repetição e Strings

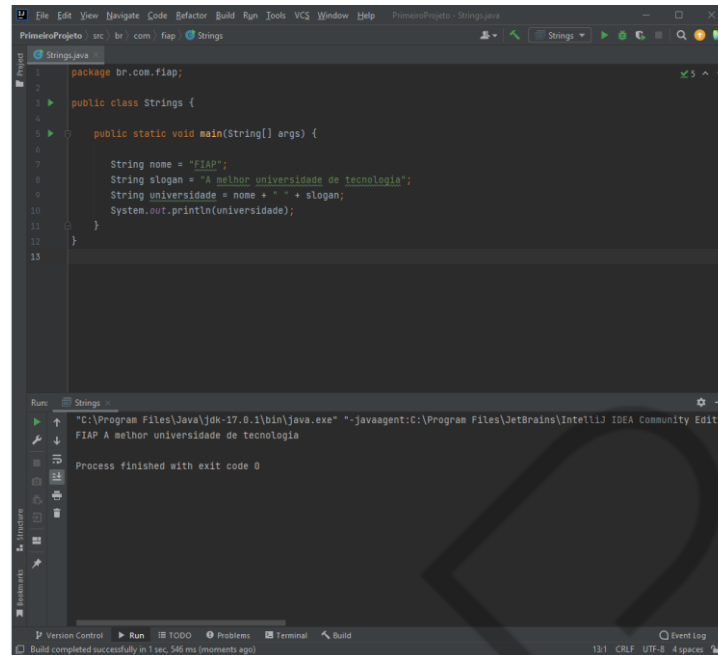


Figura 15 – Exemplo de concatenação de strings (2)

Fonte: Elaborado pelo autor (2022)

Podemos concatenar qualquer quantidade de strings e adicionar as sequências de escape, conforme o exemplo abaixo.

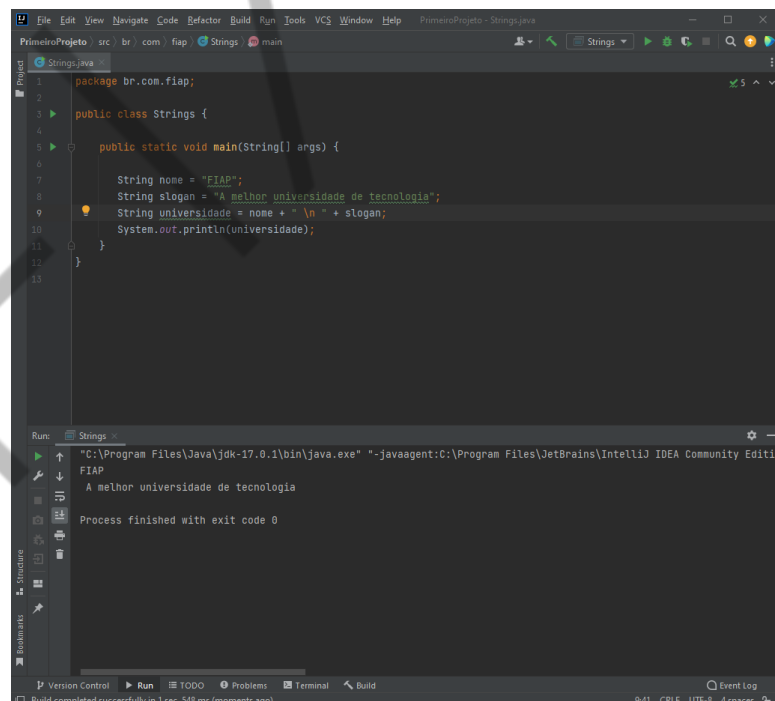


Figura 16 – Exemplo de concatenação de strings (3)

Fonte: Elaborado pelo autor (2022)

Estrutura de repetição e Strings

A seguir, veremos os principais métodos da classe **String**. Um dos principais métodos são os de comparação. Anteriormente, comparamos valores com o operador “==”, porém com Strings devemos tomar cuidado, pois não funcionam em todos os casos.

Por isso, é recomendado utilizar os métodos:

- **.equals(valor)** – Verifica se os valores das strings são iguais.
- **.equalsIgnoreCase(valor)** – Verifica se os valores das strings são iguais sem diferenciar as letras maiúsculas e minúsculas.

Mas por que utilizar os métodos e não o operador de igualdade?

O operador “==” comparará o endereço de memória em que a string está alocada e não o valor que ela possui. Analise o exemplo a seguir:

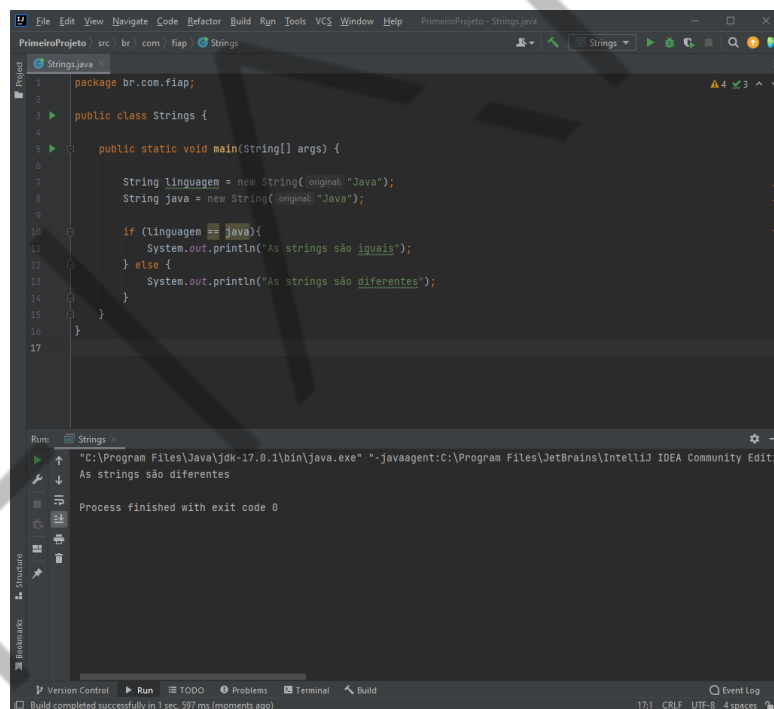


Figura 17 – Exemplo de comparação de strings (1)
Fonte: Elaborado pelo autor (2022)

No exemplo, podemos ver o resultado da execução: “As strings são diferentes”. Apesar de as duas variáveis (**linguagem** e **java**) possuírem o mesmo valor (“Java”), elas estão alocadas em endereços de memória diferentes. E o operador == compara o endereço de memória e não o valor da variável. Isso aconteceu porque o operador **new** está alocando um espaço na memória para armazenar o objeto String. E isso aconteceu duas vezes, uma para cada variável.

A Figura “Ilustração do armazenamento das strings na memória” exibe as variáveis armazenando o valor “Java”, porém cada um usando um espaço de memória diferente.

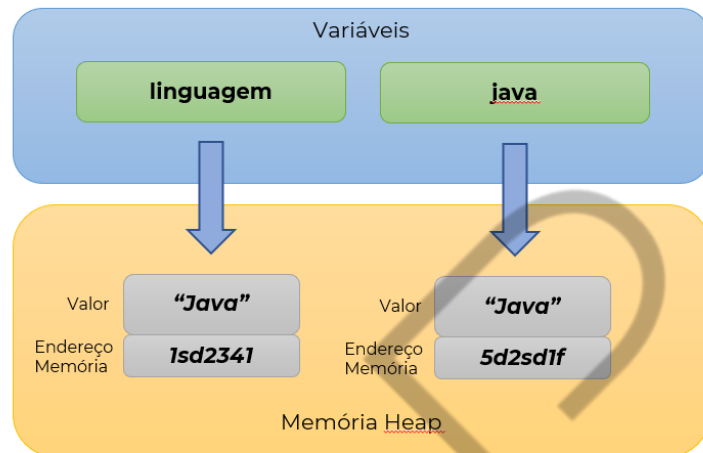


Figura 18 – Ilustração do armazenamento das strings na memória
Fonte: Elaborado pelo autor (2022)

Mas falamos que o operador `==` nem sempre funciona. Isso quer dizer que, em alguns casos, ele funciona! Isso mesmo, se inicializar a string sem o operador **new**, o valor será alocado em um pool de strings. E se uma string com o valor igual for adicionada nesse pool, o Java não armazenará o valor repetido, mas, sim, fará com que a variável de valor repetido aponte para o mesmo endereço do valor já armazenado. Dessa forma, as duas variáveis compartilharão o mesmo endereço de memória.

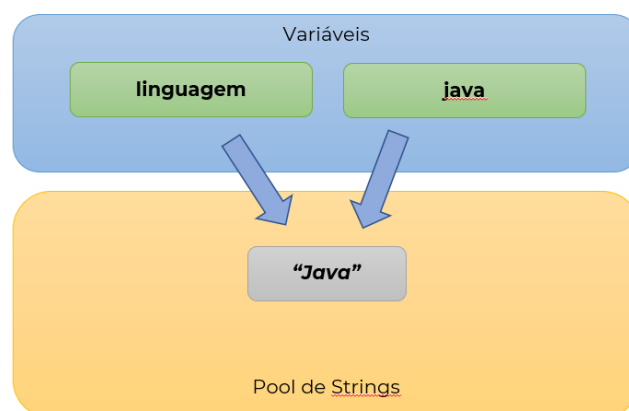
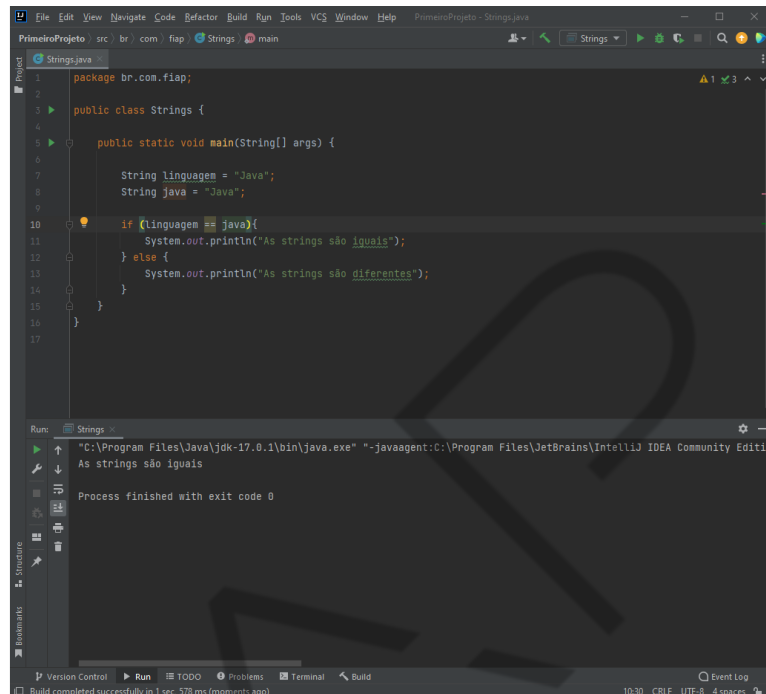


Figura 19 – Ilustração do armazenamento das strings no pool de strings
Fonte: Elaborado pelo autor (2022)

Estrutura de repetição e Strings

O exemplo a seguir apresenta o resultado “As strings são iguais”, pois compartilham o mesmo endereço de memória.



```
1 package br.com.fiap;
2
3 public class Strings {
4
5     public static void main(String[] args) {
6
7         String linguagem = "Java";
8         String java = "Java";
9
10        if (linguagem == java) {
11            System.out.println("As strings são iguais");
12        } else {
13            System.out.println("As strings são diferentes");
14        }
15    }
16 }
17
```

Run: Strings

"C:\Program Files\Java\jdk-17.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition\lib\idea_rt.jar=1273.0:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition\bin" -Dfile.encoding=UTF-8

As strings são iguais

Process finished with exit code 0

Figura 20 – Exemplo de comparação de strings (2)
Fonte: Elaborado pelo autor (2022)

Para não depender da forma como as variáveis foram inicializadas, sempre vamos utilizar os métodos apropriados para comparar o valor de duas strings.

O método equals(“valor”) compara o conteúdo de duas strings, diferenciando os caracteres maiúsculos e minúsculos. Dessa forma, a string “Java” e “java” são diferentes. O exemplo abaixo apresenta o uso do método equals.

Estrutura de repetição e Strings

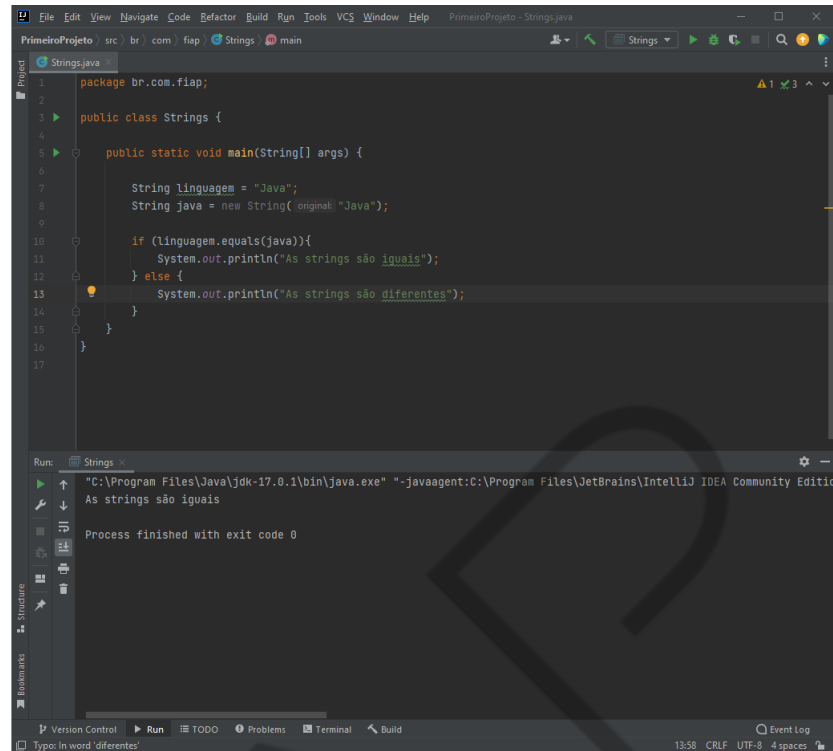


Figura 21 – Exemplo de comparação de strings (3)

Fonte: Elaborado pelo autor (2022)

Para verificar se as strings são diferentes, não é preciso um novo método. Basta utilizar o operador de negação (!) para negar a expressão e, como resultado, testar se as strings possuem valores diferentes:

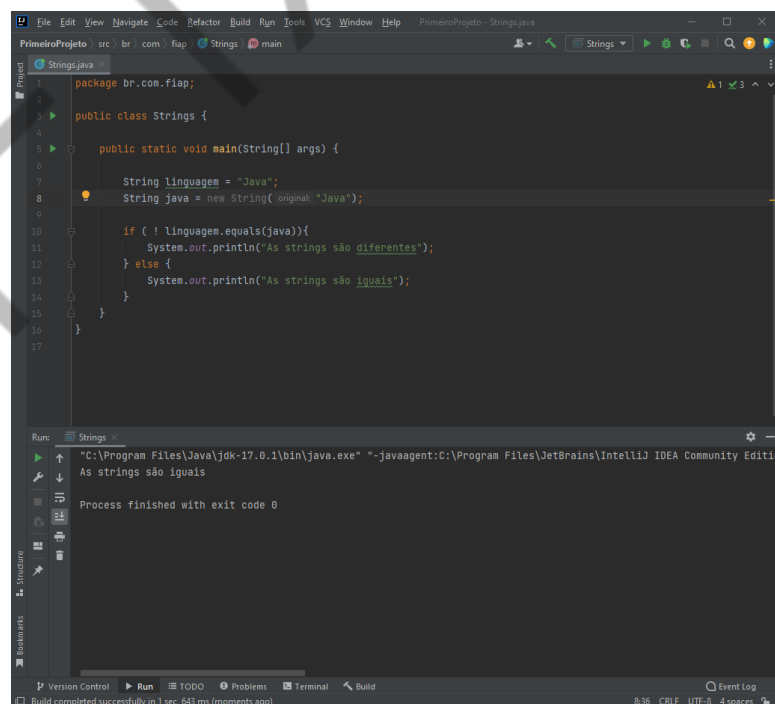
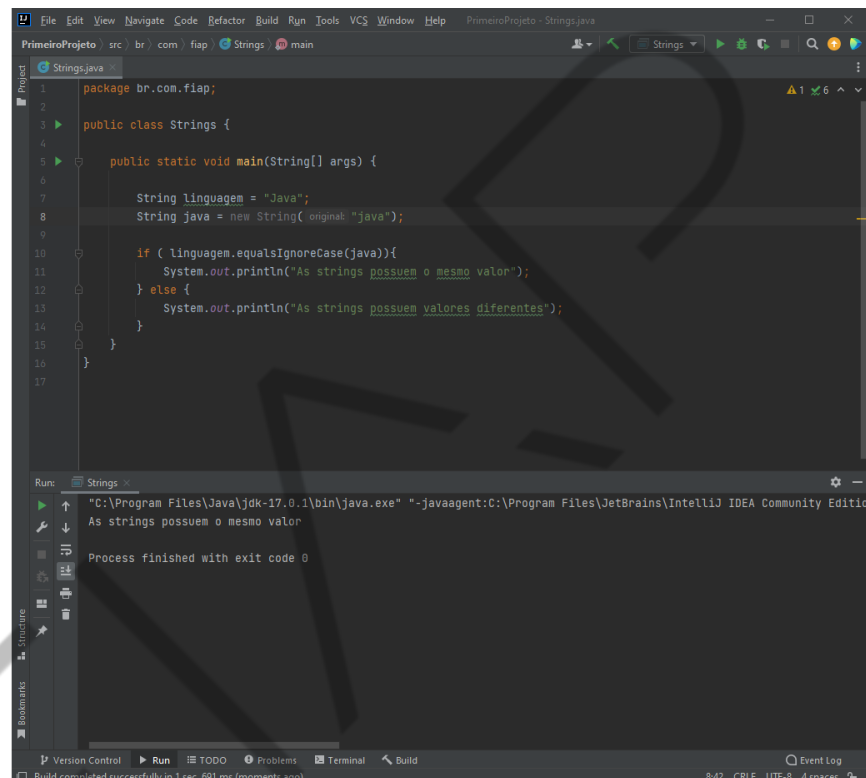


Figura 22 – Exemplo de comparação de strings (4)

Fonte: Elaborado pelo autor (2022)

Estrutura de repetição e Strings

A linguagem Java é *case sensitive*, ou seja, as letras maiúsculas e minúsculas são consideradas diferentes. Dessa forma, se compararmos a string “Java” com outra string “java”, o resultado será **false**. Neste caso, podemos usar o método `equalsIgnoreCase(valor)`, que ignora o *case sensitive*, ou seja, não diferencia as letras maiúsculas das minúsculas. Assim, o resultado do exemplo abaixo será: “As strings possuem o mesmo valor”.



```
1 package br.com.fiap;
2
3 public class Strings {
4
5     public static void main(String[] args) {
6
7         String linguagem = "Java";
8         String java = new String( original: "java");
9
10        if ( linguagem.equalsIgnoreCase(java)){
11            System.out.println("As strings possuem o mesmo valor");
12        } else {
13            System.out.println("As strings possuem valores diferentes");
14        }
15    }
16 }
17
```

Run: Strings x

```
"C:\Program Files\Java\jdk-17.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Editio
As strings possuem o mesmo valor
Process finished with exit code 0
```

Figura 23 – Exemplo de comparação de strings (5)
Fonte: Elaborado pelo autor (2022)

O operador de negação também pode ser utilizado neste método. Na verdade, esse operador pode ser utilizado em qualquer método que retorne um *boolean* ou em uma expressão que resulte em um *boolean*.

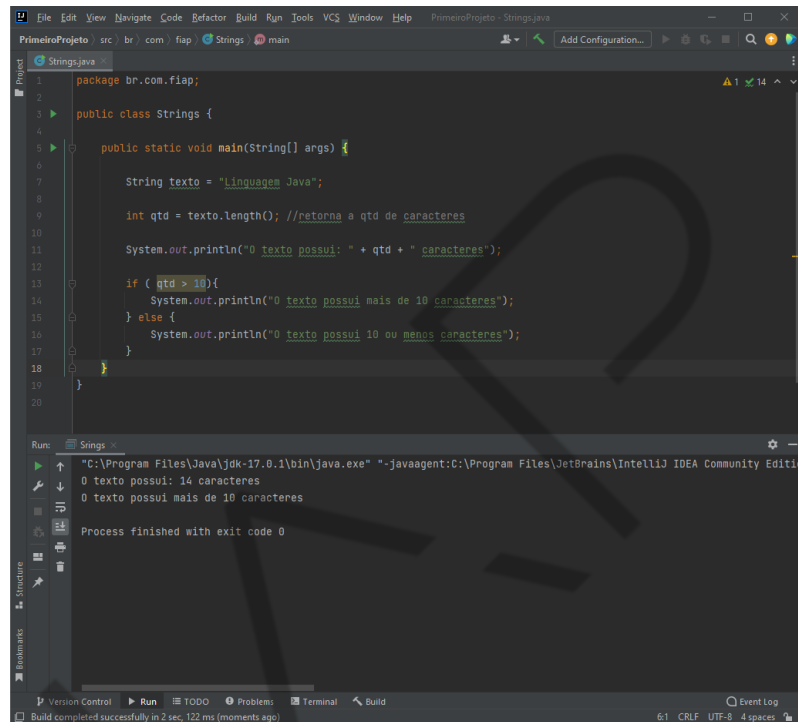
Aprendemos a comparar as strings com os métodos `equals()` e `equalsIgnoreCase()`, mas e se for preciso manipular a string ou realizar outros tipos de comparação? A classe `String` possui vários métodos para nos ajudar e veremos os principais a seguir.

Vamos supor que seja preciso validar a quantidade de caracteres de uma string, seja para limitar o tamanho de uma mensagem em uma rede social, seja a

Estrutura de repetição e Strings

quantidade de caracteres para um CPF ou CNPJ. A classe String possui o método **length()**, que retorna um número inteiro, indicando a quantidade de caracteres.

O exemplo a seguir exibe a quantidade de caracteres da string armazenada na variável **texto** e valida se possui mais de dez caracteres.



```
1 package br.com.fiap;
2
3 public class Strings {
4
5     public static void main(String[] args) {
6
7         String texto = "Linguagem Java";
8
9         int qtd = texto.length(); //retorna a qtd de caracteres
10
11         System.out.println("O texto possui: " + qtd + " caracteres");
12
13         if (qtd > 10){
14             System.out.println("O texto possui mais de 10 caracteres");
15         } else {
16             System.out.println("O texto possui 10 ou menos caracteres");
17         }
18     }
19 }
20
```

Run: Strings

```
"C:\Program Files\Java\jdk-17.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition\lib\idea_rt.jar=6270:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition\bin" -Dfile.encoding=UTF-8
0 texto possui: 14 caracteres
0 texto possui mais de 10 caracteres
Process finished with exit code 0
```

Figura 24 – Exemplo recuperando a quantidade de caractere de uma string
Fonte: Elaborado pelo autor (2022)

Outro método muito útil é o **indexOf()**, que permite localizar a primeira ocorrência de um caractere ou uma palavra em uma string. Dessa forma, se for localizado o caractere ou a palavra, o método retorna à posição (index) onde a palavra ou o caractere está localizado. Caso não encontre, o valor -1 é retornado, indicando que não possui o valor procurado.

Por exemplo, vamos declarar uma variável do tipo string com o valor “FIAP A MELHOR”. Se utilizarmos o método para retornar o index da primeira ocorrência do caractere “A” (lembre-se de que letras maiúsculas e minúsculas são diferentes), o resultado será 2.

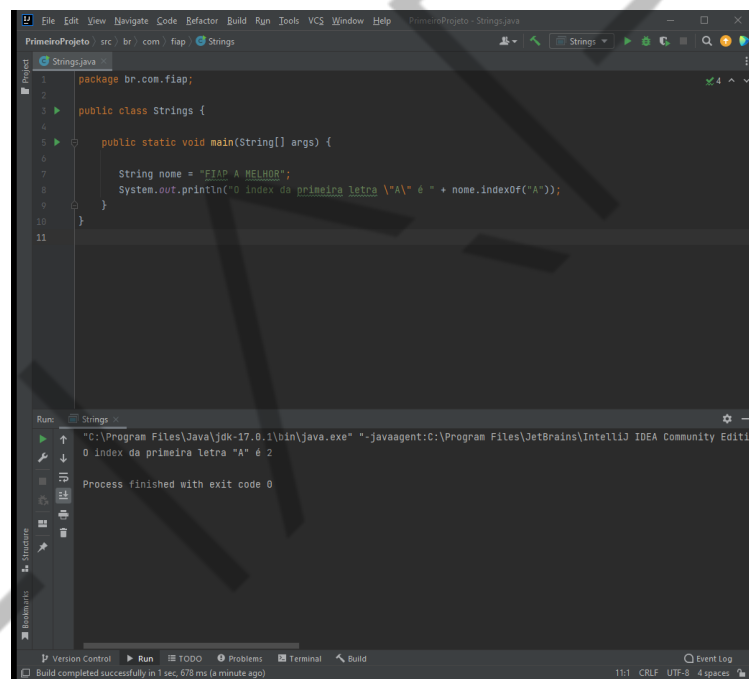
Estrutura de repetição e Strings

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
String	F	I	A	P		A		M	E	L	H	O	R

Figura 25 – Index dos caracteres de uma string
Fonte: Elaborado pelo autor (2022)

Observe que o index começa da posição 0 e os espaços em branco também são considerados.

A implementação de um exemplo é apresentada na Figura “Exemplo da utilização do método indexOf” a seguir.



```
package br.com.fiap;

public class Strings {

    public static void main(String[] args) {

        String nome = "FIAP A MELHOR";
        System.out.println("O index da primeira letra 'A' é " + nome.indexOf('A'));
    }
}
```

Run: Strings

"C:\Program Files\Java\jdk-17.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition\lib\idea_rt.jar=1100:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition\bin" -Dfile.encoding=UTF-8

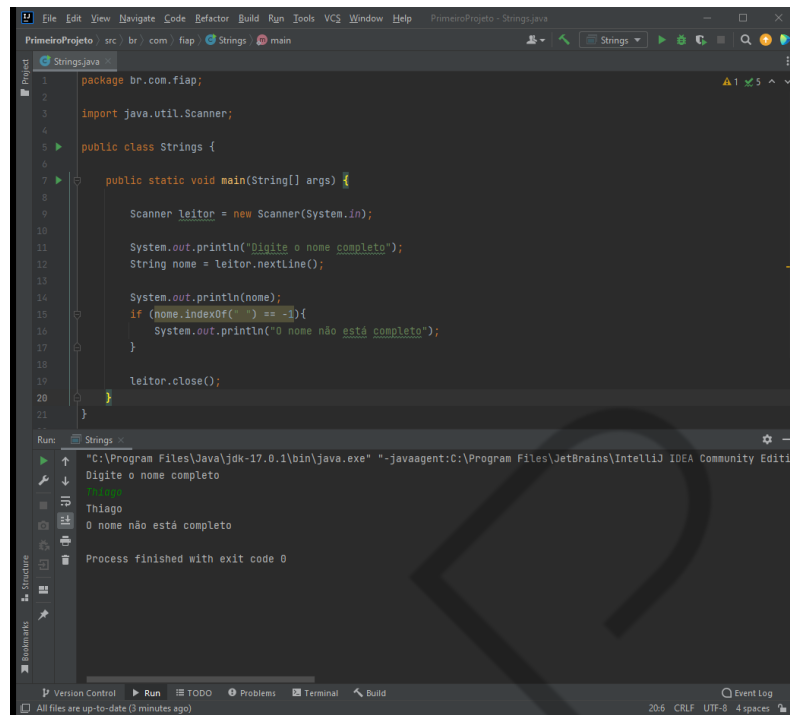
0 index da primeira letra "A" é 2

Process finished with exit code 0

Figura 26 – Exemplo da utilização do método indexOf
Fonte: Elaborado pelo autor (2022)

Com o método **indexOf()**, podemos verificar, por exemplo, se uma string possui mais de uma palavra e se existe algum espaço em branco. Por exemplo, vamos implementar uma aplicação para o usuário inserir o nome completo. Depois, exibiremos o nome e analisaremos se o valor informado possui pelo menos duas palavras, ou seja, se o `indexOf(" ")` é igual a `-1`. Assim, verificamos se a string possui pelo menos um espaço em branco.

Estrutura de repetição e Strings



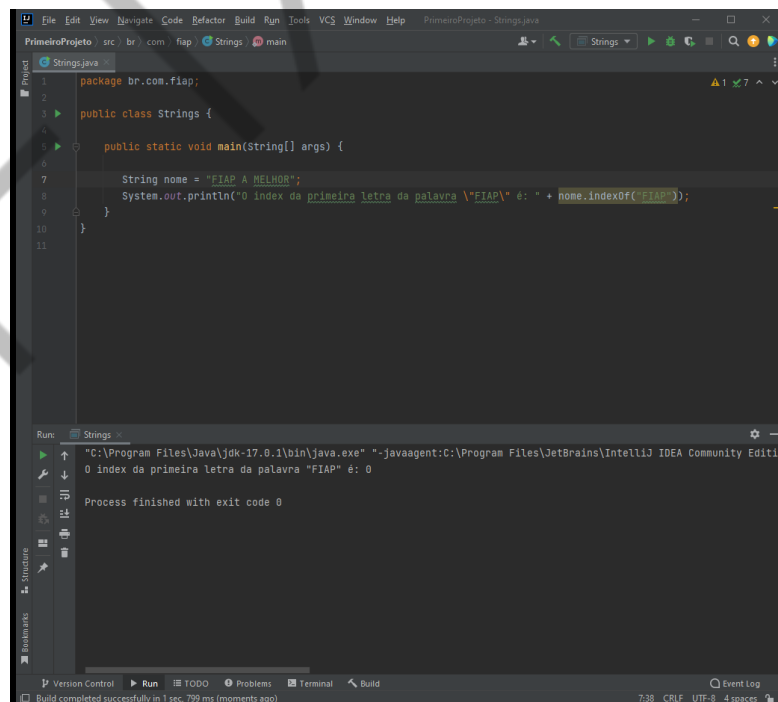
```
1 package br.com.fiap;
2
3 import java.util.Scanner;
4
5 public class Strings {
6
7     public static void main(String[] args) {
8
9         Scanner leitor = new Scanner(System.in);
10
11         System.out.println("Digite o nome completo");
12         String nome = leitor.nextLine();
13
14         System.out.println(nome);
15         if (nome.indexOf(" ") == -1){
16             System.out.println("O nome não está completo");
17         }
18
19         leitor.close();
20     }
21 }
```

Run: Strings

```
"C:\Program Files\Java\jdk-17.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Editio
Digite o nome completo
Thiago
O nome não está completo
Process finished with exit code 0
```

Figura 27 – Exemplo de verificação de nome completo
Fonte: Elaborado pelo autor (2022)

O método **indexOf()** também pode ser usado para procurar por uma sequência de caracteres. Para isso, basta informar uma palavra em vez do caractere, simples assim.



```
1 package br.com.fiap;
2
3 public class Strings {
4
5     public static void main(String[] args) {
6
7         String nome = "FIAP A MELHOR";
8         System.out.println("O index da primeira letra da palavra 'FIAP' é: " + nome.indexOf("FIAP"));
9     }
10 }
11 }
```

Run: Strings

```
"C:\Program Files\Java\jdk-17.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Editio
O index da primeira letra da palavra "FIAP" é: 0
Process finished with exit code 0
```

Figura 28 – Exemplo do método indexOf utilizando uma palavra
Fonte: Elaborado pelo autor (2022)

O método **lastIndexOf()** funciona da mesma forma do método **indexOf()**, com a diferença de retornar o index da última ocorrência encontrada ao invés da primeira. Também funciona com um caractere ou um conjunto de caracteres.

Por exemplo, se utilizarmos o método **lastIndexOf("A")** na string "FIAP A MELHOR", o valor retornado será 5, a posição da última letra "A".

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
String	F	I	A	P		A		M	E	L	H	O	R

Figura 29 – Exemplo do método **lastIndexOf**
Fonte: Elaborado pelo autor (2022)

Os próximos métodos manipulam a string, ou seja, podemos modificar os caracteres, recuperar parte de uma string, deixar todas as letras maiúsculas ou minúsculas. Mas lembre-se de que a string é imutável, ou seja, não é possível modificar o seu valor. Assim, esses métodos irão criar uma nova string como resultado e não modificar a string existente.

Para recuperar uma string a partir de um trecho de outra string, podemos usar o método **substring**. Esse método recebe duas informações: a posição inicial (incluindo) e a posição final (excluindo) do conjunto de caracteres a serem copiados da string original. O caractere da posição inicial será incluído na nova string. Já o caractere da posição final não será incluído.

Por exemplo, se utilizarmos o método **substring(1,8)** na string "FIAP A MELHOR", a parte da string retornada será: "IAP A M", conforme podemos ver na Figura "Exemplo do método **substring**".

		Início (incluindo)				Fim (excluindo)							
		↓				↓							
Index	0	1	2	3	4	5	6	7	8	9	10	11	12
String	F	I	A	P		A		M	E	L	H	O	R

Figura 30 – Exemplo do método **substring**
Fonte: Elaborado pelo autor (2022)

Vamos ver, na prática, com a implementação do exemplo:

Estrutura de repetição e Strings

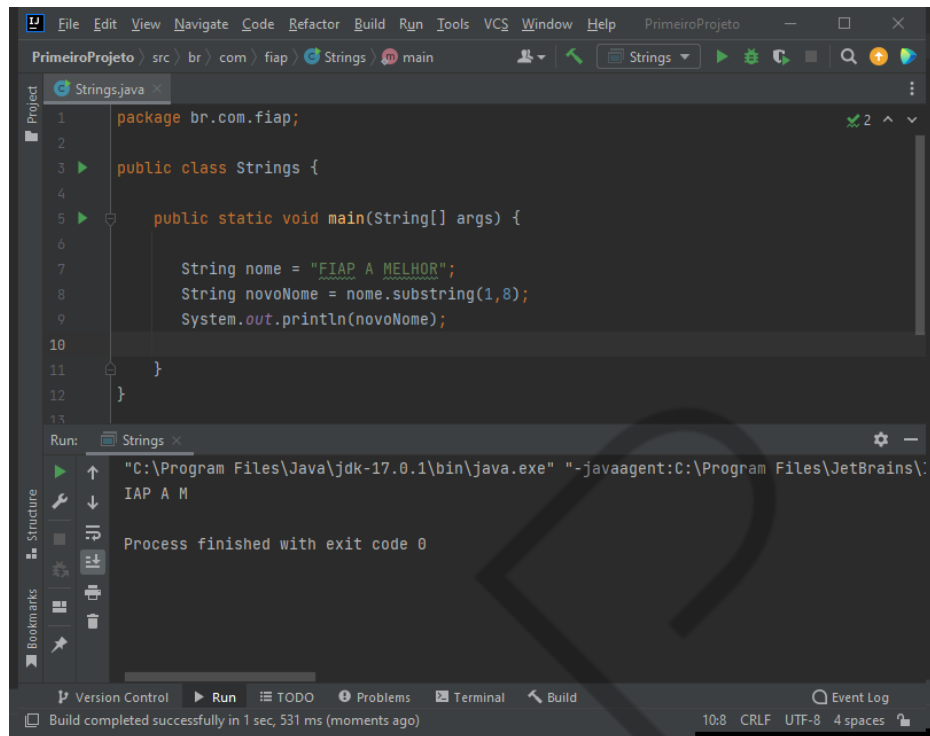


Figura 31 – Implementação do exemplo do método subString

Fonte: Elaborado pelo autor (2022)

O método `subString` também pode receber somente um parâmetro, que indicará a posição inicial do conjunto de caracteres que serão copiados para a nova string. Assim, a nova string será criada a partir da posição informada até o final.

Desse modo, se o método `subString(6)` for chamado na mesma string “FIAP A MELHOR”, a nova string criada será “MELHOR”.

								Início (incluindo)					Até o final
								↓					↓
Index	0	1	2	3	4	5	6	7	8	9	10	11	12
String	F	I	A	P		A		M	E	L	H	O	R

Figura 32 – Método subString com um parâmetro

Fonte: Elaborado pelo autor (2022)

A implementação é apresentada na imagem abaixo.

Estrutura de repetição e Strings

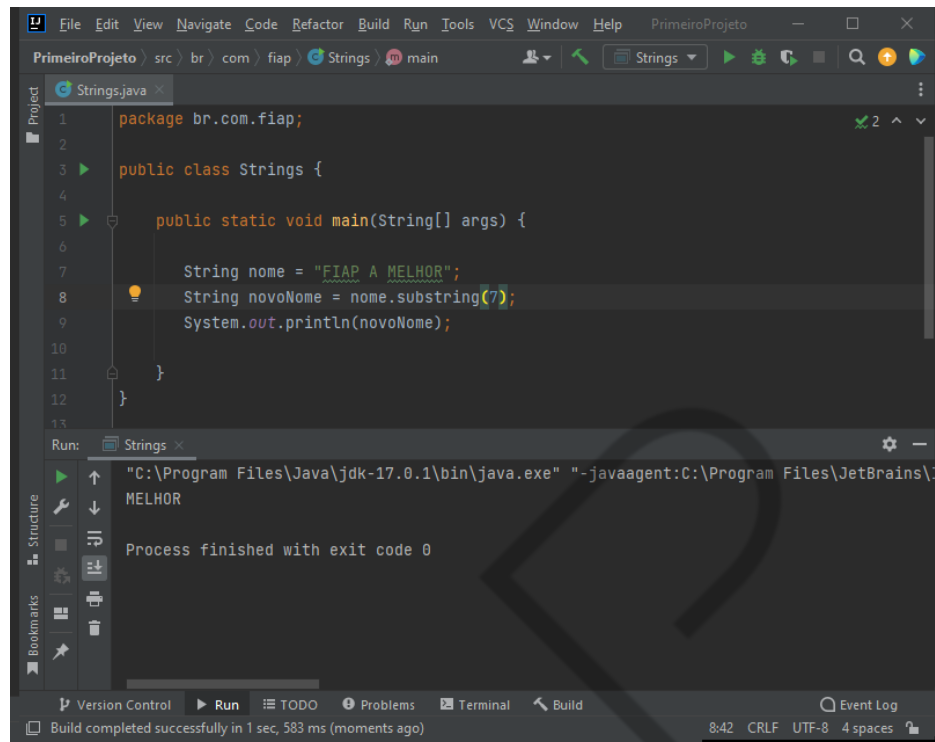


Figura 33 – Implementação do exemplo do método subString com um parâmetro
Fonte: Elaborado pelo autor (2022)

Os métodos podem ser aplicados em conjunto, como, por exemplo, utilizar o método `indexOf` para localizar um caractere e utilizar a sua posição para ser o ponto inicial ou final da nova string obtida pelo método `subString`. Assim, por exemplo, podemos separar o nome e o sobrenome de uma pessoa localizando o primeiro espaço em branco na string e usar a sua posição para obter substrings: primeiro o nome, da posição 0 até a posição do espaço em branco; depois, o sobrenome, da posição do espaço em branco mais 1, até o final da string. Como pode ser observado no exemplo abaixo.

Estrutura de repetição e Strings

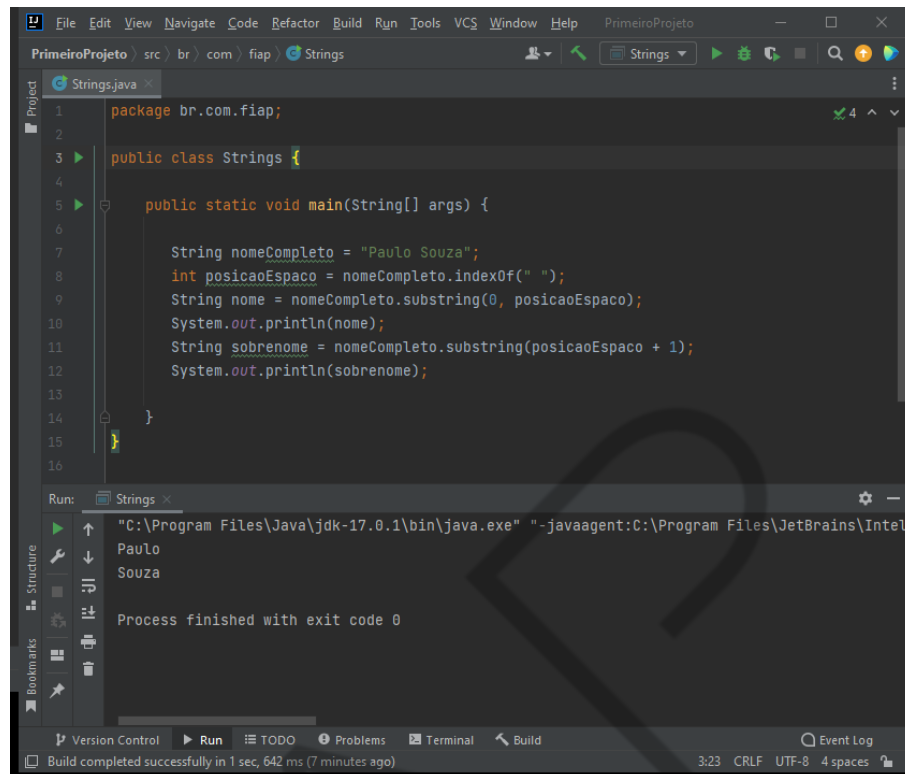


Figura 34 – Implementação do exemplo com o método subString e indexOf
Fonte: Elaborado pelo autor (2022)

A classe String possui os métodos toUpperCase e toLowerCase, que transformam todos os caracteres em letras maiúsculas e minúsculas, respectivamente.

Lembre-se de que os métodos de manipulação de string criam outras strings. Assim, no exemplo a seguir, podemos ver que os métodos transformaram a string, deixando todos os caracteres maiúsculos e minúsculos, associando a novas variáveis. Porém, a string que está na variável original se manteve.

Estrutura de repetição e Strings

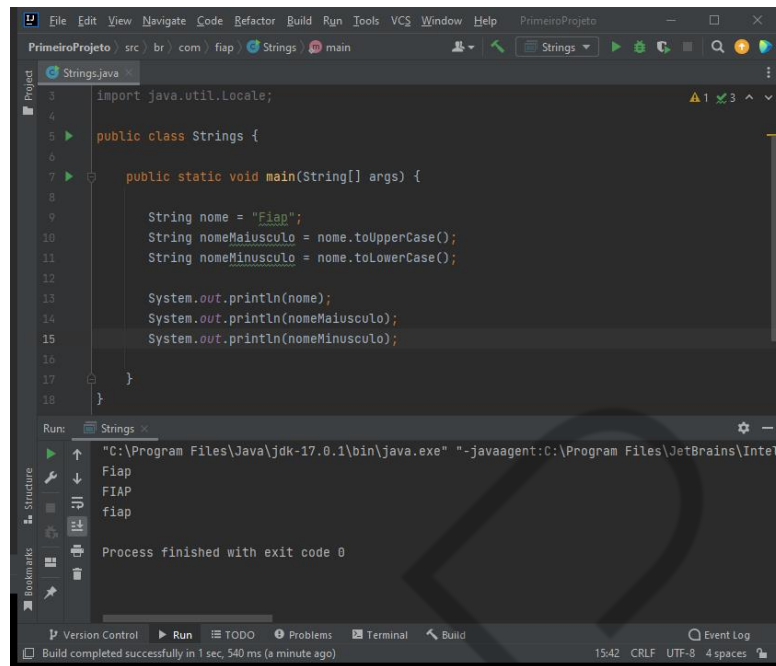
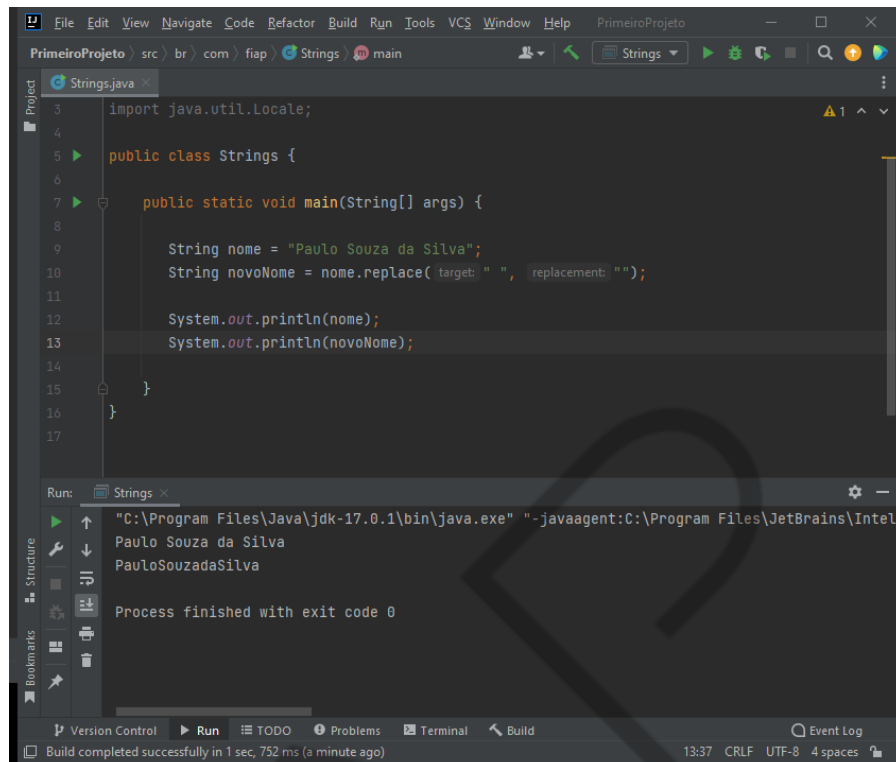


Figura 35 – Implementação do exemplo com os métodos `toUpperCase` e `toLowerCase`
Fonte: Elaborado pelo autor (2022)

É possível substituir os caracteres ou até mesmo as palavras de uma string, criando uma nova string. O método `replace` recebe como parâmetros o caractere ou a palavra a ser substituída e a letra ou palavra para substituir, de forma muito parecida com os editores de texto mais conhecidos.

Assim, podemos substituir qualquer caractere ou conjunto de caracteres por outros caracteres ou até mesmo por nenhum. No exemplo, estamos substituindo o espaço em branco " " por "", ou seja, estamos retirando os espaços em branco da string.

Estrutura de repetição e Strings



```
3 import java.util.Locale;
4
5 public class Strings {
6
7     public static void main(String[] args) {
8
9         String nome = "Paulo Souza da Silva";
10        String novoNome = nome.replace(" ", "");
11
12        System.out.println(nome);
13        System.out.println(novoNome);
14
15    }
16 }
17
```

Run: Strings x

```
"C:\Program Files\Java\jdk-17.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA\bin\idea-agent.jar" -Didea.config.path=C:\Program Files\JetBrains\IntelliJ IDEA\config -Didea.copyright.path=C:\Program Files\JetBrains\IntelliJ IDEA\copyright Paulo Souza da Silva
PauloSouzadaSilva

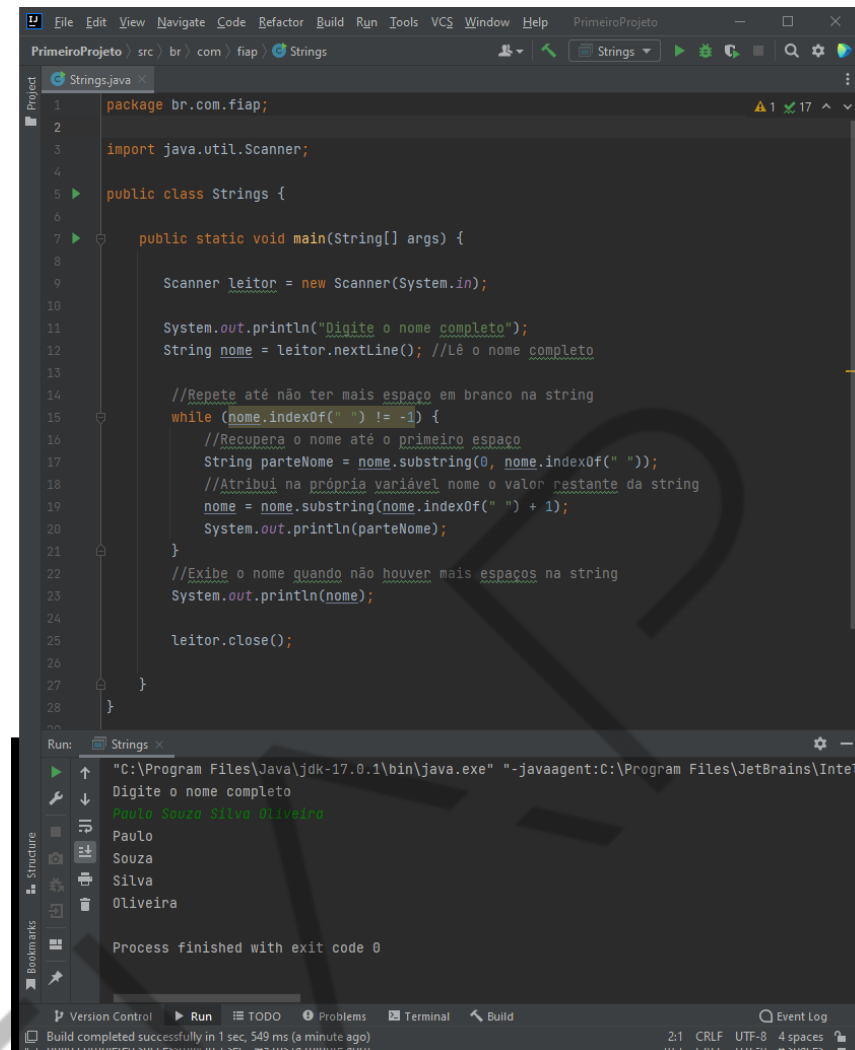
Process finished with exit code 0
```

Build completed successfully in 1 sec, 752 ms (a minute ago) 13:37 CRLF UTF-8 4 spaces

Figura 36 – Implementação do exemplo com o método replace
Fonte: Elaborado pelo autor (2022)

Para finalizar, vamos evoluir o programa que separa o nome completo de uma pessoa, pensando que o nome pode ser composto por mais de dois nomes. Assim, por exemplo, para o nome “Paulo Souza Silva Oliveira”, vamos separar os nomes “Paulo”, “Souza”, “Silva”, “Oliveira”. Porém, o programa deve ler o nome da pessoa e exibir os nomes separadamente. Deixe o programa esperto para que também funcione caso o usuário informe somente um nome.

Estrutura de repetição e Strings



```
1 package br.com.fiap;
2
3 import java.util.Scanner;
4
5 public class Strings {
6
7     public static void main(String[] args) {
8
9         Scanner leitor = new Scanner(System.in);
10
11         System.out.println("Digite o nome completo");
12         String nome = leitor.nextLine(); // Lê o nome completo
13
14         // Repete até não ter mais espaço em branco na string
15         while (nome.indexOf(" ") != -1) {
16             // Recupera o nome até o primeiro espaço
17             String parteNome = nome.substring(0, nome.indexOf(" "));
18             // Atribui na própria variável nome o valor restante da string
19             nome = nome.substring(nome.indexOf(" ") + 1);
20             System.out.println(parteNome);
21         }
22         // Exibe o nome quando não houver mais espaços na string
23         System.out.println(nome);
24
25         leitor.close();
26     }
27 }
28 }
```

Run: Strings

"C:\Program Files\Java\jdk-17.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\Intel
Digite o nome completo
Paulo Souza Silva Oliveira
Paulo
Souza
Silva
Oliveira
Process finished with exit code 0

Figura 37 – Implementação do exercício proposto
Fonte: Elaborado pelo autor (2022)

Uma solução possível para o problema foi apresentado. Na implementação, podemos ver, na linha 9, a declaração da variável responsável por ler os dados inseridos pelo usuário; depois, na linha 11 e 12, exibimos uma mensagem para o usuário pedindo para inserir o nome completo e recebemos o valor digitado.

Na linha 15, temos um laço de repetição que será utilizado para repetir o código até que a string não contenha mais espaços em branco. Dentro do laço, separamos a primeira parte do nome, na linha 17, e depois atualizamos o valor da variável nome com o restante da string (linha 19). Por fim, exibimos o nome que foi recuperado.

Fora do laço, precisamos imprimir o nome que sobrou, ou seja, o último nome, caso o usuário tenha informado vários nomes, ou o único nome informado. Já que se o usuário inserir somente um nome, o laço de repetição não será executado.

REFERÊNCIAS

BARNES, David J. **Programação Orientada a Objetos com Java**: uma introdução prática utilizando Blue J. São Paulo: Pearson, 2004.

COLHO, Alex. **Java com Orientação a Objetos**. Rio de Janeiro: Ciência Moderna, 2012.

DEITEL, Paul; DEITEL, Harvey. **Java como programar**. 10. ed. São Paulo: Pearson, 2016.

HORSTMANN, Cay; CORNELL, Gary. **Core Java – Volume I – Fundamentos**. 8. ed. São Paulo: Pearson 2009.

SIERRA, Kathy; BATES, Bert. **Use a cabeça! Java**. Rio de Janeiro: Alta Books, 2010.